



실습 중심 강의

Coding Agent 기본

AI를 활용한 SW 개발 기초

PRD → GitHub → 배포 → 모니터링 · 프롬프트 · 컨텍스트 · 하네스 엔지니어링을 직접 따라 하는 실습

개발자 트랙

LinkSnip — 풀스택 URL 단축 서비스 배포·운영

비개발자 트랙

Tetris — 프롬프트 복붙으로 게임 만들어 배포

개념은 함께, 실습은 각자 트랙으로 — 이 슬라이드 자료는 두 대상을 한 강의에서 안내합니다.

이 강의의 전체 여정

공통 · 개발자 + 비개발자

함께 배우는 개념 → 트랙별 실습 → 다시 함께 마무리

① 공통 개념

Part 0-6
Codex 지도 · 프롬프트 · 컨텍스트 · 하
네스



② 트랙별 실습

개발자

비개발자

개발자 Part 7-13 / 비개발자 Part 7-8
각자 다른 제품을 직접 배포



③ 공통 마무리

회고 · 치트시트 · 부록
핵심 개념을 내 언어로 정리

하나의 실제 제품을 처음부터 만들어 인터넷에 배포하고 운영까지 경험합니다. 대상에 따라 만드는 제품만 달라집니다.

슬라이드 색 표기 약속



공통 — 개발자·비개발자 모두



개발자 트랙 전용 실습



비개발자 트랙 전용 실습

두 개의 트랙 — 무엇이 같고, 무엇이 다른가

공통 · 개발자 + 비개발자

이 강의의 핵심 안내: 개념은 공유하고 실습만 갈립니다

함께 배우는 공통 개념 (Part 0 – 6)

계정·환경 준비 · Codex 기술 지도 · 프롬프트 엔지니어링 · 컨텍스트 엔지니어링 · AGENTS.md/Sandbox/Approvals · 하네스 엔지니어링

↓ 여기서부터 실습이 갈립니다 ↓

개발자 트랙

LinkSnip · URL 단축 웹 서비스

만드는 것	실제 배포·운영되는 풀스택 웹 서비스
기술 스택	Vite + TypeScript · Cloudflare Pages Functions · Workers KV
추가 실습	OpenRouter 연동 · Git/GitHub · Cloudflare 배포 · GA4·Clarity
범위	Part 7–13 + 보너스(Tetris)

비개발자 트랙

Tetris · 브라우저 게임

만드는 것	코딩 경험 0에서도 완주하는 웹 게임
기술 스택	순수 HTML / CSS / JavaScript (빌드·프레임워크 없음)
작업 방식	준비된 프롬프트를 복사·붙여넣기만 하면 진행
범위	Part 7–8 (GitHub Pages 배포)

워크북 사용법 · 표기 약속

공통 · 개발자 + 비개발자

강의 슬라이드 흐름을 그대로 손으로 따라 하는 실습 교재

회색 코드 박스

터미널에 입력하거나 파일에 붙여넣는 실제 명령·코드. 그대로 복사해 사용

[NOTE]

알아두면 좋은 보충 설명

[TIP]

더 빠르게·안전하게 하는 요령

[주의]

데이터 손실·요금 발생 등 조심할 부분

▶ 실습

직접 따라 하는 단계 — 결과를 확인하고 넘어가기

✓ 체크

그 단계를 끝냈는지 스스로 점검하는 항목

모든 명령·코드·프롬프트를 복사해 쓸 수 있게 수록되어 있어, 강사 없이 혼자서도 끝까지 따라 할 수 있습니다.

이 강의를 끝내면 할 수 있는 것

학습 목표 · Learning Outcomes

- 1 Codex의 사용 방식·실행 환경·지시/권한 체계 등 전체 기술 지도를 설명할 수 있다.
- 2 Prompt·Context·Harness와 AGENTS.md로 프로젝트 규칙과 검증 체계를 구성할 수 있다.
- 3 Sandbox·Approvals로 안전하게 실행하고 config.toml 기초를 설정할 수 있다.
- 4 PRD → GitHub → 배포까지 하나의 제품을 실제로 출시·운영할 수 있다.

4번 목표의 '제품'은 트랙에 따라 다릅니다 — 개발자는 LinkSnip, 비개발자는 Tetris.

Part 0. 시작 전 준비 — 계정과 환경

공통 · 개발자 + 비개발자

실습 중 막히지 않도록 계정·도구를 먼저 준비 (약 20-30분)

만들어 둘 계정 (모두 무료로 시작)

ChatGPT / OpenAI

Codex 로그인·모델 사용 — chatgpt.com

GitHub

소스 코드 저장·배포 트리거 — github.com

Cloudflare · 비개발자 트랙에선 생략 가능
웹 배포 (개발자 트랙에서 사용)

로컬 개발 도구 설치

필요한 것은 Node.js와 Git 뿐입니다.

설치 여부 확인

```
node -v
```

```
git --version
```

Node.js 18 이상 (20 LTS 권장)

사전 점검 체크리스트

✅ ChatGPT/OpenAI 로그인 가능

✅ GitHub 로그인됨

✅ node -v 버전 출력

✅ git --version 버전 출력

Part 1. Codex 전체 기술 지도 — 네 개의 축

공통 · 개발자 + 비개발자

Codex는 터미널·에디터에서 실제로 코드를 읽고 고치고 실행하는 코딩 에이전트

1 모델

코드를 생성·추론하는 AI 엔진
(gpt-5.x-codex 계열)

다루는 곳 **Part 2**

2 사용 방식

Codex를 '어디서' 쓰는가 —
App · IDE 확장 · CLI

다루는 곳 **Part 2**

3 지시·권한

AGENTS.md(규칙) · Sandbox(
범위) · Approvals(승인)

다루는 곳 **Part 5**

4 하네스

빌드·테스트·배포·로그로 결과
를 '검증'하는 틀

다루는 곳 **Part 6**

이 네 축이 결과 품질을 좌우합니다: 무엇을 시키는가(프롬프트) · 무엇을 보여주는가(컨텍스트) · 어떤 검증 틀에서 일하게 하는가(하네스).

Part 1. 핵심 용어 미리 보기

공통 · 개발자 + 비개발자

지금 다 외울 필요는 없습니다 — 각 용어는 해당 파트에서 실습으로 익힙니다

Prompt

Codex에게 시키는 자연어 지시. 작게 쪼개고 검증 방법을 함께 준다

Context

Codex가 참고하는 정보(파일·이미지·대화·AGENTS.md). 모델의 '작업 기억'

AGENTS.md

프로젝트 상시 규칙 문서. 모든 작업 전에 Codex가 먼저 읽는다

Harness

작업을 감싸는 검증·안전 장치 (빌드/테스트/린트/샌드박스)

Sandbox

Codex가 파일·네트워크를 쓸 수 있는 '기술적 범위'

Approvals

위험한 행동 전에 사람에게 물어보게 하는 '정책'

Thread

하나의 작업 세션. 여러 프롬프트가 이어진 대화

MCP

Codex에 외부 도구(GitHub 등)를 연결하는 표준 규약

Part 2. 사용 방식 — App · IDE · CLI

공통 · 개발자 + 비개발자

어떤 방식으로 쓰든 실제로 일하는 AI 모델은 똑같습니다

App

MAIN

독립 데스크톱/웹 앱. 인앱 브라우저·변경
미리보기·승인 UI·클라우드 작업

언제 쓰나 **이 과정의 메인 실습**

IDE 확장

VS Code 등 에디터에 통합. 열린 파일·선택
영역을 자동 컨텍스트화

언제 쓰나 **코드 작성 중 즉시 도움**

CLI

터미널 기반. 스크립트/CI 자동화에 적합
(Rust로 작성되어 가볍고 빠름)

언제 쓰나 **자동화·보조**

[NOTE] 왜 App을 메인으로? App은 Codex가 제안한 파일 변경을 diff(전/후 비교)로 보여주고, 승인·미리보기·클라우드 작업을 한 화면에서 다룰 수 있어 입문자에게 가장 안전합니다.

Part 2. ▶ 실습 — App 설치부터 첫 작업까지

공통 · 개발자 + 비개발자

설치 → 폴더 열기 → 첫 지시 → 결과 확인

1 설치 & 로그인

openai.com에서 Codex 데스크톱 앱 설치 → 같은 ChatGPT 계정으로 로그인. 기본값은 '작업 폴더 안에서만 변경, 위험한 동작은 승인'

2 첫 프로젝트 열기

빈 폴더(예: codex-hello)를 만들고 App에서 '폴더 열기'. 이 폴더가 Codex의 작업 공간이 된다

3 첫 작업 — 인사하기

Codex App 작성 칸

폴더에 hello.txt 파일을 만들고
"Hello Codex" 한 줄을 써줘.

→ 파일 목록에서 hello.txt 를 열어
내용이 "Hello Codex"인지 확인

이어지는 실습(2-4): 다양한 일 시켜 보기

· 엑셀 VLOOKUP 예시 만들기

· 다운로드 폴더 정리 방법

· CSV 파일이 어떤 데이터인지 설명

· 이미지 5장 생성해 저장

Part 2-6. Codex App 사이드바 핵심 기능

공통 · 개발자 + 비개발자

웹 만들어 공유 · 반복 작업 자동화 · 기능 확장



사이트 (Sites)

프롬프트 한 줄로 웹페이지·앱·게임을 만들어 OpenAI 서버에 바로 호스팅. 미리보기
→ 공개 범위 선택 → 링크 공유

Codex에게 맡기는 배포



예약 (Scheduled)

'나중에·정기적으로' 할 일을 맡기기. 매일/매주 반복, 변화 감시. 실행 결과는 예약 화면에 수신함처럼 모임

반복 작업 자동화



플러그인 (Plugins)

새 기능(스킬·앱·템플릿)을 더하는 확장 장터. 문서·PDF·스프레드시트·프레젠테이션이 기본 포함

기능 확장

[TIP] 세 기능은 서로 연결됩니다 — 예약 작업이 플러그인·스킬을 함께 호출할 수 있습니다.

Part 3. 프롬프트 엔지니어링 — 나쁨 vs 좋음

공통 · 개발자 + 비개발자

Codex는 받은 만큼 일합니다

× 나쁜 프롬프트

로그인 기능 만들어줘.

대상 파일 · 동작 규칙 · 검증 방법이 모두 빠짐

→ Codex가 알아서 추측 → 결과가 매번 달라진다

✓ 좋은 프롬프트

src/auth.ts 에 이메일+비밀번호 로그인을 구현하고,
실패 시 에러 반환. 완료 후 npm test로 검증해줘.

① 대상 파일 지정 ② 동작 규칙 ③ 반환 형태 ④ 검증 방법
→ 결과가 예측 가능하고, Codex가 스스로 검증까지

좋은 프롬프트의 두 가지 핵심 원칙

→ 이 둘을 매번 챙기는 체크리스트가 P·C·T·F

1 작업을 작고 명확하게 쪼갬다

큰 작업은 '먼저 계획을 세우라'고 시킨 뒤 한 단계씩 진행

2 '끝났는지 어떻게 아는지'를 함께 준다

테스트·실행·린트 등 스스로 검증할 방법을 포함

Part 3. 재사용 템플릿 — P · C · T · F

공통 · 개발자 + 비개발자

네 글자만 외우면 거의 모든 작업에 적용되는 체크리스트 (순서 아님)



Persona / Role 역할·관점

어떤 전문가의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"



Context 배경·제약

건드릴 파일 · 지켜야 할 규칙 · 참고 자료

예) @src/auth.ts 수정 · 비밀번호 평문 저장 금지



Task 할 일·검증

구체적으로 무엇을 하고, 성공을 어떻게 확인하는가

예) 로그인 구현 → 실패 시 에러 반환 → npm test 통과



Format 산출물 형태

결과를 어떤 형태로 받을지 정한다

예) 변경 요약과 diff로 보여줘

네 조각을 이어 붙이면 곧 12쪽의 '좋은 프롬프트' — 필요 없는 요소는 건너뛰어도 됩니다.

Part 3. 핵심 기법 — 결과를 바꾸는 6가지 도구

공통 · 개발자 + 비개발자

P·C·T·F에 없으면 결과가 확 달라지는, 검증된 프롬프트 기법 (필요한 것만 골라 쓰기)

1

역할 부여 Role · Persona

누구의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"

2

예시 제공 Few-shot

원하는 형식·패턴을 예시로 보여준다

예) 입력→출력 한 쌍을 붙여 출력 형식을 고정

3

계획 먼저 Plan · 단계적 사고

바로 고치지 말고 계획부터 세우게 한다

예) "수정 전에 먼저 계획을 제안해줘 (read-only)"

4

작게 쪼개기 To-Do

큰 일을 한 항목씩 체크리스트로 나눈다

예) "이 To-Do를 한 항목씩, 한 커밋씩 진행"

5

검증 지시 Verification

끝났는지 아는 법을 함께 준다

예) "구현 후 npm test 통과까지 확인"

6

반복 개선 Iterate

한 번에 X — 대화로 다듬는다

예) 안 되면 증상을 구체화하거나 새 스레드로

Part 3. ▶ 실습 — 나쁜 한 줄을 좋은 프롬프트로

공통 · 개발자 + 비개발자

P·C·T·F와 핵심 기법을 세 가지 상황에 직접 적용해 봅니다

1

① 계획부터

공통

PRD로 To-Do 뽑기

✗ "테트리스 만들어줘"

✓ @PRD.md 를 읽고 — 수정하지 말고(read-only) — Tetris MVP 완성을 위한 To-Do를 체크리스트로 먼저 만들어 줘.

2

② 구현+검증

개발자

한 기능을 검증까지

✗ "로그인 기능 만들어줘"

✓ 꼼꼼한 TypeScript 개발자로서 @src/auth.ts 에 이메일+비밀번호 로그인을 구현. 실패 시 에러 반환, 평문 저장 금지. 완료 후 npm test 통과를 확인하고 변경 요약과 diff로 보여줘.

3

③ 디버깅

비개발자

증상을 구체적으로

✗ "게임이 안 돼요, 고쳐줘"

✓ 블록이 좌우 벽을 통과해요. 벽 경계에서 멈추도록 충돌 검사를 고치고, 고친 뒤 브라우저에서 좌우 이동이 정상 인지 확인하는 방법을 알려줘.

세 예시 모두 12·13쪽의 두 원칙과 P·C·T·F가 그대로 적용됩니다 — 대상·규칙·검증을 빠짐없이.

Part 4. 컨텍스트 엔지니어링 — 네 가지 전략

공통 · 개발자 + 비개발자

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'

핵심 비유 — LLM은 컴퓨터, 컨텍스트 윈도우는 작업 기억(RAM). '많이 넣을수록 똑똑'은 오해 — 정확히, 관련 있는 것만.

● 작성 (Write)

중요한 규칙·결정을 밖에 적어 두기

→ AGENTS.md에 저장 (Part 5)

● 선택 (Select)

지금 작업에 필요한 것만 골라 넣기

→ @로 파일 쿼리하기

● 압축 (Compress)

길어진 대화를 요약해 자리 확보

→ 컴팩션 · 스레드 나누기

● 격리 (Isolate)

작업별로 맥락을 분리하기

→ 한 작업 = 한 스레드

Part 5. AGENTS.md — 작업 전에 먼저 읽는 규칙서

공통 · 개발자 + 비개발자

프로젝트의 상시 규칙(컨텍스트) · 계층적으로 합쳐짐

1

전역 (Global)

~/ .codex/AGENTS.md

내 모든 프로젝트 공통 개인 규칙

2

프로젝트

저장소 루트/AGENTS.md

이 프로젝트의 셋업·검증 규칙

3

하위 폴더

services/x/AGENTS.md

특정 폴더에만 다르게 적용



우선순위: 전역 → 프로젝트 루트 → 현재 폴더 (가까울수록 우선)

이 파일이 곧 '컨텍스트'이자 '하네스'

'변경 시 반드시 lint-test를 통과' 같은 규칙을 적어 두면, 매번 말하지 않아도 Codex가 스스로 지킵니다.

▶ 실습

- 전역 AGENTS.md 만들기
- 프로젝트 AGENTS.md 만들기
- 로드되는지 확인 요청하기

config.toml — 모델·승인·샌드박스 기본값 저장
(~/ .codex/config.toml)

Part 5. Sandbox & Approvals — 안전하게 실행

공통 · 개발자 + 비개발자

샌드박스 = '무엇을 할 수 있나(범위)' · 승인 = '언제 물어보나(시점)'

읽기 전용

```
sandbox = read-only
```

수정 없이 분석·계획만. 가장 안전

가장 안전

Auto (권장)

```
workspace-write + on-request
```

작업 폴더 내 자동, 폴더 밖·네트워크는 승인

균형 · 권장

Full access

```
danger-full-access + never
```

제한 없음. 격리된 VM/CI에서만

위험 · 격리 환경

네트워크는 기본 차단(보안). 패키지 설치처럼 필요할 때만 '이번에만 허용'을 누릅니다. App은 승인 모드 버튼 한 번으로 전환.

Part 6. 하네스 엔지니어링 — 결과 검증의 틀

공통 · 개발자 + 비개발자

Codex가 만든 결과가 진짜 동작하는지 자동으로 확인하게 만드는 안전망

1

빌드 (Build)

코드가 실제로 컴파일/번들되는
가

`npm run build / typecheck`

2

테스트 (Test)

기능이 의도대로 동작하는가

`npm test (Vitest)`

3

배포 (Deploy)

산출물을 실제로 배포할 수 있는
가

GitHub Pages · Cloudflare

4

로그 (Log)

실행 결과·에러를 확인할 수 있는
가

테스트/런타임 로그

하네스 = '검증 가능한 규칙' + '권한 경계'. 일부러 코드를 깨보고(회귀 테스트), 빨간불이 났다가 수정 후 다시 통과하는 흐름을 직접 보면 '안전망'의 감이 옵니다.

여기까지가 공통 개념입니다

이제 실습이 갑니다

같은 원리(프롬프트·컨텍스트·하네스)를 쓰지만, 만드는 제품이 다릅니다.

개발자 트랙

LinkSnip — 풀스택 URL 단축 서비스

Part 7-13 · OpenRouter · Cloudflare · GA4/Clarity

비개발자 트랙

Tetris — 브라우저 게임

Part 7-8 · 프롬프트 복붙 · GitHub Pages 배포

실습 프로젝트 한눈에 비교

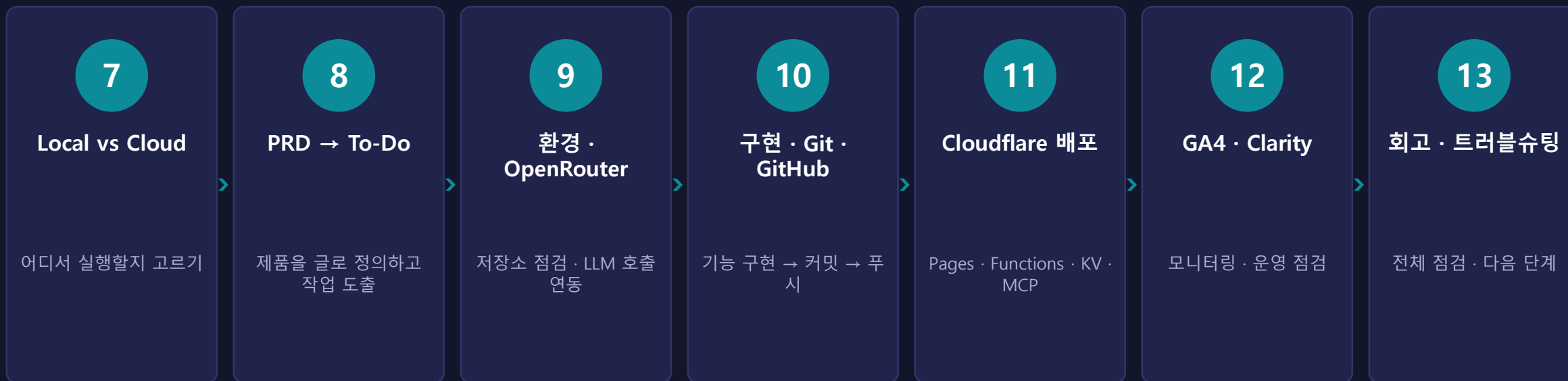
공통 · 개발자 + 비개발자

같은 학습 원리, 다른 결과물 — 본인 트랙을 확인하세요

구분	개발자 트랙	비개발자 트랙
만드는 제품	LinkSnip · URL 단축 서비스	Tetris · 브라우저 게임
코딩 사전지식	필요 (TypeScript·CLI 익숙)	불필요 (프롬프트 복붙)
기술 스택	Vite+TS · CF Functions · KV	순수 HTML/CSS/JS
외부 연동	OpenRouter (LLM 호출)	없음
배포 방식	Cloudflare Pages (자동)	GitHub Pages
모니터링	GA4 · Microsoft Clarity	(선택) 없음
워크북 범위	Part 7-13 + 보너스	Part 7-8

LinkSnip 실습 로드맵

Part 7 → 13 · '작게 만들고, 통과하면 바로 커밋한다'



다음 슬라이드부터 각 단계를 자세히 봅니다.

개발자 · Part 8 — PRD → To-Do 도출

개발자 트랙

코드를 짜기 전에 '무엇을 왜 만드는가'를 한 페이지로 (LinkSnip)

① PRD.md 작성

PRD = Product Requirements Document (제품 요구사항 정의서). PRD가 좋으면 Codex 결과도 좋아집니다.

```
# PRD - LinkSnip
```

```
## 문제
```

```
긴 URL을 짧게 · 클릭 통계
```

```
## 핵심 기능(MVP)
```

```
F1 링크 생성 / F2 리다이렉트  
F3 클릭 집계 / F4 통계 보기
```

② To-Do 도출 (읽기 전용 모드)

@PRD.md 이 PRD를 읽고, MVP를 완성하기 위한 To-Do 목록을 만들어줘. 수정은 하지 마.

[TIP] 계획·분석 단계는 *read-only*로 받으면 파일이 바뀔 걱정이 없습니다.

✅ 이 단계의 완료 조건

- PRD.md를 저장했다
- @로 PRD를 참조해 To-Do를 받았다
- 첫 작업 한 항목만 골라 진행 준비

개발자 · Part 9-10 — 연동 · 구현 · GitHub

개발자 트랙

OpenRouter로 LLM 호출 → 기능 구현 → 커밋 → 푸시

● Part 9 · OpenRouter 연동

- 여러 회사 모델을 하나의 API 주소로 호출하는 중개 서비스
- API 키는 .env에 저장하고 .gitignore로 보호 (절대 코드·GitHub에 올리지 않기)
- src/lib/ai.ts 에 suggestTitle(url) 같은 호출 모듈을 Codex로 생성

● Part 10 · 구현 · Git · GitHub

- PRD/To-Do를 따라 한 항목씩 구현 — 한 To-Do = 한 커밋
- 변경 미리보기(diff) 확인 → git add → 커밋(요약+이유)
- git push로 GitHub에 올리면 자동 배포·클라우드 실행이 가능해짐

[주의] API 키를 소스나 GitHub에 올리면 도용·과금 위험 — 반드시 .env + .gitignore.

개발자 · Part 11 — Cloudflare 배포

개발자 트랙

정적 프론트엔드 + API/리다이렉트 + 저장소를 무료 한도로 운영

Cloudflare Pages

정적 프론트엔드 호스팅

Pages Functions

API · 리다이렉트 (Workers)

Workers KV

'코드 → URL' 저장소

★ 입문자 권장

방식 A · GitHub 연결 자동 배포

- Workers & Pages → Create → Connect to Git
- linksnip 저장소 선택 · 빌드 설정(Vite)
- push할 때마다 자동 재배포 (...pages.dev)

세밀한 제어가 필요할 때. KV·Functions를 직접 붙이고 로컬 통합 미리보기.

방식 B · Wrangler CLI (수동)

```
npm install -D wrangler

npm run build

# 로컬에서 Functions까지 미리보기
```

개발자 · Part 12 — GA4 · Clarity 모니터링

개발자 트랙

배포 후엔 '사람들이 실제로 어떻게 쓰는지' 봅니다

Google Analytics 4

방문·이벤트 같은 '숫자'를 봅니다.

측정 ID(G-XXXX) 발급 → 모든 html <head>에 gtag 스크립트 삽입

Microsoft Clarity

히트맵·세션 녹화 같은 '행동'을 봅니다.

추적 스크립트를 모든 html에 삽입 → 커밋 → push하면 자동 재배포

운영 점검 — 무엇을 보나

방문/실시간

GA4 실시간

✓ 내 테스트 접속이 즉시 잡힌다

핵심 이벤트

GA4 이벤트

✓ shorten_success 가 기록된다

막힘 지점

Clarity 녹화

✓ 복사·줄이기 버튼이 잘 눌린다

에러/리다이렉트

Cloudflare 로그

✓ 404·5xx 비율이 낮다

테트리스 만들기 로드맵

Part 7 → 8 · 프로그래밍 경험 0에서도 프롬프트 복붙만으로 완주

7

PRD → To-Do

Tetris를 글로 정의

8.1-8.4

게임 만들기

보드·낙하·조작·점수를 프롬프트로

8.5

실행·테스트

시나리오 점검·디버깅

8.6

GitHub 올리기

커밋·푸시 (버튼으로도 가능)

8.7

GitHub Pages

누구나 접속하는 링크 배포

각 '프롬프트' 상자의 내용을 그대로 복사해 Codex 작성 칸에 붙여넣기만 하면 됩니다.

비개발자 · Part 7 — Tetris PRD → To-Do

비개발자 트랙

게임도 '무엇을 만들지' 글로 먼저 정의합니다

① PRD.md 작성

아래 내용을 그대로 붙여넣어 Tetris/PRD.md 로 저장하게 시킵니다.

```
# PRD - Tetris
```

```
## 목표
```

```
브라우저에서 도는 테트리스
```

```
## 핵심 기능
```

```
블록 낙하 · 회전 · 이동  
줄 삭제 · 점수 · 게임오버
```

② To-Do 도출 (읽기 전용)

@PRD.md 이 PRD를 읽고 Tetris MVP를 완성하기 위한 To-Do 목록을 만들어줘. 수정은 하지 마.

[TIP] 계획 단계는 read-only로 받으면 파일이 안 바뀌어 안전합니다.

이 흐름은 개발자 트랙과 똑같습니다

PRD를 쓰고 → @로 참조해 To-Do를 받는 방법은 공통. 만드는 제품(게임 vs 서비스)만 다릅니다.

비개발자 · Part 8 — 프롬프트로 게임 만들기

비개발자 트랙

프롬프트 0 → 7 을 순서대로 복사·붙여넣기

0

프로젝트 골격

순수 HTML/CSS/JS로 빈 보드·점수 영역

2

자동 낙하 & 충돌

0.5초마다 한 칸 내려오고 바닥에서 고정

4

라인 클리어 & 점수

가득 찬 줄 삭제 → 위 블록 내리고 점수 증가

6

다듬기 (선택)

다음 블록 미리보기·레벨·속도 증가

1

보드 & 블록

그리드와 테트로미노(블록 모양) 정의·렌더링

3

키보드 조작

←→ 이동, ↓ 빨리, ↑ 회전, Space 즉시 낙하

5

게임 오버 & 재시작

새 블록 공간 없으면 종료·다시 시작

7

테스트 점검

시나리오 점검하고 안 되는 항목 고치기

[TIP] 문제가 생기면 증상을 구체적으로 적어 디버깅 프롬프트로 (예: "블록이 벽을 통과해요").

비개발자 · Part 8.6–8.7 — 배포하기

비개발자 트랙

만든 게임을 누구나 접속하는 링크로 무료 공개

1

Git으로 기록

코드가 동작하면 버전을 기록(commit). VS Code '소스 제어' 패널에서 버튼으로도 가능 — 세 단어만: add → commit → push

2

GitHub에 올리기

GitHub CLI(gh)로 인증·저장소 생성이 쉬움. 또는 웹에서 빈 저장소를 먼저 만들고 push

3

GitHub Pages 배포

저장소 Settings → Pages 에서 공개. 정적 파일(HTML/CSS/JS)이라 무료 배포 가능 → github.io 링크 생성

✅ **완료 조건** 블록 낙하·조작·줄 삭제·점수·게임오버가 모두 동작하고, GitHub Pages 주소로 접속된다.

다시 함께 — 마무리 회고 & 부록

공통 · 개발자 + 비개발자

두 트랙 모두 같은 자료로 정리하고 복습합니다

마무리 회고 (Part 13 / 공통)

- 전체 흐름 한눈에 보기
- 핵심 개념을 '한 문장으로' 말해 보기
- 최종 점검 체크리스트
- 자주 막히는 문제 트러블슈팅

부록 — 언제든지 꺼내 쓰는 참고

- A 프롬프트·컨텍스트·하네스 치트시트
- B config.toml 전체 예시(주석 포함)
- C AGENTS.md 템플릿(복사용)
- D 용어집

한 문장으로 말해 보기 — 개념 회고

프롬프트

작게 쪼개고 검증 방법을 함께 준다

컨텍스트

필요한 것만, 정확히 (많이 ≠ 똑똑)

AGENTS.md

작업 전 먼저 읽는 상시 규칙서

하네스

결과가 진짜 동작하는지 검증하는 안
전망

강사용 — 혼합 교실 운영 가이드

공통 · 개발자 + 비개발자

개발자·비개발자가 한 강의에 함께 있을 때

● 공통 구간 (Part 0-6)

- 전원 함께 진행 · 예시 프롬프트는 LinkSnip 소재로 통일되어 있음
- 개념마다 '왜 중요한가'를 한 문장으로 정리하고 넘어가기

● 분기 지점 (트랙 안내)

- 슬라이드 색 규칙(파랑=공통 · 청록=개발자 · 주황=비개발자) 먼저 공지
- 코딩 사전지식 기준으로 스스로 트랙을 선택하게 안내

● 실습 구간

- 가능하면조를 나눠 병렬 진행 (개발자실 / 비개발자실)
- 인원이 적으면 비개발자(Tetris) 완주 → 개발자(LinkSnip) 시연 순서 권장
- 비개발자 막힘 시: 프롬프트 복붙 확인 · 디버깅 프롬프트 유도

● 합류 (마무리)

- 부록·치트시트로 전원 복습 · 개념 회고는 트랙 무관 동일 질문 사용

핵심 한 장 요약

이것만 기억하면 됩니다

Codex는 에이전트다

챗봇이 아니라 파일을 직접 고치고 테스트까지 돌린다

프롬프트

작게 쪼개고, 검증 방법을 함께 준다

컨텍스트

많이가 아니라 '정확히, 필요한 것만'

규칙 & 안전

AGENTS.md에 규칙, Sandbox/Approvals로 안전

하네스

빌드·테스트·배포·로그로 결과를 검증

실습은 트랙별

개발자=LinkSnip 배포·운영 / 비개발자=Tetris 게임



수고하셨습니다

개념은 함께, 실습은 각자 — 이제 내 손으로 만든 결과물이 인터넷에 있습니다.

다음 단계

- 부록 치트시트로 P.C.T.F를 내 프로젝트에 적용해 보기
- AGENTS.md에 나만의 '완료의 정의' 못 박기
- 개발자: 기능 추가·도메인 연결 / 비개발자: LinkSnip 트랙 도전

Q & A · 질문을 환영합니다